

Agentic AI-Based Travel Planning Assistant Using LangChain

Abstract

The "Agentic AI-Based Travel Planning Assistant Using LangChain" is an intelligent AI-powered travel assistant designed to automate and personalize travel planning. The system uses Large Language Models (LLMs), LangChain framework, Gemini API, and custom tools to help users plan trips efficiently.

The assistant can recommend flights, hotels, tourist attractions, weather forecasts, and estimated budgets based on user preferences. Unlike traditional chatbots, this project uses an agentic AI architecture where the AI model reasons about the user query, selects appropriate tools, executes them, and generates intelligent responses.

The project demonstrates the practical implementation of Generative AI, tool calling, prompt engineering, and AI agents in the tourism and travel domain.

Introduction

Travel planning is often time-consuming because users need to search multiple platforms for flights, hotels, weather updates, local attractions, and budget estimation. Traditional travel applications provide information separately, requiring significant manual effort.

With advancements in Artificial Intelligence and Large Language Models (LLMs), it is possible to create intelligent assistants capable of automating the entire travel planning process.

This project focuses on building an Agentic AI Travel Assistant using LangChain and Gemini API. The assistant behaves like an intelligent travel planner that can:

- Understand natural language queries
- Analyze user travel requirements
- Select relevant tools automatically
- Generate personalized travel recommendations
- Estimate travel costs
- Provide interactive responses

The system combines AI reasoning with external tool execution to provide a smart and dynamic travel planning experience.

Objectives

The main objectives of this project are:

1. To develop an AI-powered travel planning assistant.
2. To automate flight and hotel recommendations.
3. To provide weather forecasts and tourist attraction suggestions.
4. To estimate travel budgets intelligently.
5. To implement agentic AI architecture using LangChain.
6. To integrate Gemini API with custom tools.
7. To build an interactive user interface using Streamlit.

Problem Statement

Users often face difficulties while planning trips because:

- Travel information is scattered across multiple websites.
- Manual searching consumes time.
- Budget estimation is complicated.
- Personalized recommendations are limited.
- Existing systems lack intelligent reasoning.

The proposed AI-based travel assistant solves these problems by providing a unified and intelligent travel planning system.

Technologies Used

Technology	Purpose
Python	Core programming language
LangChain	AI agent orchestration
Gemini API	Large Language Model
Streamlit	User Interface
dotenv	Environment variable management
Google Generative AI	AI model integration
Custom Tools	Flight, hotel, weather, budget, places

Project Structure

The project follows a modular architecture for better scalability and maintainability.

Folder Structure

```
AGENTIC AI-BASED TRAVEL PLANNING ASSISTANT
├── data/
│   ├── flights.json
│   ├── hotels.json
│   └── places.json
├── tools/
│   ├── flight_tool.py
│   ├── hotel_tool.py
│   ├── places_tool.py
│   ├── weather_tool.py
│   └── budget_tool.py
├── env/
├── .env
├── agent.py
├── app.py
├── requirements.txt
└── README.md
```

Module Description

File	Purpose
app.py	Streamlit frontend application
agent.py	LangChain agent configuration
flight_tool.py	Flight recommendation logic
hotel_tool.py	Hotel filtering and recommendations
places_tool.py	Tourist attraction recommendation engine
weather_tool.py	Weather API integration
budget_tool.py	Travel cost estimation
.env	API key management
data/*.json	Static dataset storage

Technical Architecture

The project uses an Agentic AI pipeline where the Large Language Model performs reasoning and dynamically selects tools based on user queries.

Architecture Layers

1. Presentation Layer

Implemented using Streamlit.

Responsibilities:

- User interaction
- Travel preference collection
- Displaying generated itineraries
- Downloading travel plans

2. Application Layer

Implemented using LangChain Agents.

Responsibilities:

- Query interpretation
- Tool orchestration
- Multi-step reasoning
- Response synthesis

3. AI Layer

Implemented using Gemini API.

Responsibilities:

- Natural language understanding
- Tool selection reasoning
- Itinerary generation
- Conversational response generation

4. Tool Layer

Custom Python tools are used for:

- Flight recommendation
- Hotel recommendation
- Tourist place discovery
- Weather forecasting
- Budget estimation

5. Data Layer

Static JSON datasets are used for:

- Flights
 - Hotels
 - Tourist places
-

System Architecture

The architecture of the system consists of the following components:

1. User Interface
2. LangChain Agent
3. Gemini LLM
4. Custom Tools
5. Response Generation

Flow of the system:

User Query → LangChain Agent → Gemini LLM → Tool Selection → Tool Execution → Final AI Response

Working Principle

The travel assistant follows an agentic AI workflow.

Step 1: User Query

The user enters a travel-related query such as:

"Plan a 3-day trip from Delhi to Goa under ₹20,000."

Step 2: AI Reasoning

The Gemini model analyzes the request and determines which tools are required.

Step 3: Tool Selection

The LangChain agent selects appropriate tools such as:

- Flight Search Tool
- Hotel Recommendation Tool
- Places Discovery Tool
- Weather Tool
- Budget Estimation Tool

Step 4: Tool Execution

The selected tools process the request and return relevant information.

Step 5: Final Response

The AI combines all outputs and generates a complete travel plan.

Features of the System

1. Flight Recommendation

The assistant recommends flights based on:

- Source city
- Destination city
- Budget
- Travel preferences

2. Hotel Recommendation

The assistant suggests hotels using:

- Budget constraints
- Hotel rating
- Hotel type
- Destination city

3. Tourist Attraction Recommendation

The system recommends:

- Beaches
- Adventure places
- Shopping areas
- Nightlife locations
- Cultural attractions

4. Weather Forecast

Users can check weather conditions before planning trips.

5. Budget Estimation

The assistant calculates estimated expenses including:

- Flight cost
- Hotel cost
- Food expenses
- Local transport expenses

6. Natural Language Interaction

Users can communicate with the assistant using normal conversational language.

LangChain Integration

LangChain is used as the core orchestration framework.

The project uses:

- Agents
- Tools
- LLM integration
- Prompt reasoning
- Tool execution

LangChain enables the AI model to:

- Decide which tool to use
 - Execute functions dynamically
 - Handle multi-step reasoning
-

Gemini API Integration

Gemini API is used as the Large Language Model (LLM) for:

- Natural language understanding
- Decision making
- Tool reasoning
- Response generation

The project integrates Gemini using:

```
from langchain_google_genai import ChatGoogleGenerativeAI
```

The model used:

```
model="gemini-1.5-flash"
```

Tool Implementation

The project uses LangChain Tool objects to integrate external functionalities with the AI agent.

Flight Search Tool

The Flight Search Tool handles both dictionary and string-based user inputs.

Technical Features

- Dynamic query parsing
- Input validation
- Multi-format input handling
- AI-driven flight recommendation simulation

Sample Logic

```
if isinstance(query, dict):  
    source = query.get("source_city", "")  
    destination = query.get("destination_city", "")
```

Workflow

1. Extract source and destination.
 2. Validate user input.
 3. Generate simulated flight recommendations.
 4. Return formatted flight details.
-

Hotel Recommendation Tool

The Hotel Recommendation Tool filters hotel data from a local JSON dataset.

Technical Features

- JSON data processing
- Dynamic filtering
- Budget-based recommendation
- City matching logic

Sample Logic

```
with open("data/hotels.json", "r") as file:  
    hotels = json.load(file)
```

Filtering Conditions

- City matching
 - Price range filtering
 - Dynamic result generation
-

Places Discovery Tool

The Places Tool recommends tourist attractions based on user interests.

Technical Features

- Interest mapping system
- Semantic category matching
- Recommendation ranking
- Dynamic filtering

Interest Mapping Example

```
interest_mapping = {  
    "beaches": "beach",  
    "historical places": "fort",  
    "shopping": "market"  
}
```

Recommendation Pipeline

1. Load tourist places dataset.
 2. Match user interests.
 3. Filter place categories.
 4. Return top-rated places.
-

Weather Lookup Tool

The Weather Tool integrates with the Open-Meteo API.

Technical Features

- REST API integration
- Real-time weather retrieval
- JSON response handling
- HTTP request processing

API Example

```
url = f"https://api.open-meteo.com/v1/forecast"  
response = requests.get(url)
```

Budget Estimation Tool

The Budget Tool estimates overall trip expenses.

Cost Components

- Flight cost
- Hotel cost
- Food expenses
- Local transport expenses

Technical Features

- Cost aggregation
- Dynamic estimation
- Structured output formatting

Sample Logic

```
total = (  
    flight_cost  
    + hotel_cost  
    + food_cost  
    + transport_cost  
)
```

LangChain Agent Configuration

The AI agent is configured using LangChain's `initialize_agent()`.

Agent Components

- Gemini LLM
- Tool list
- ReAct reasoning strategy
- Error handling

Configuration Example

```
agent = initialize_agent(  
    tools=tools,  
    llm=llm,  
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,  
    verbose=True,  
    handle_parsing_errors=True  
)
```

Streamlit Frontend Implementation

The frontend application is developed using Streamlit.

Frontend Features

- Responsive UI
- Sidebar travel preferences
- Interactive forms
- Dynamic itinerary generation
- Downloadable travel plans

User Inputs

- Source city
- Destination city
- Budget
- Travel type
- Hotel preference
- Interests
- Food preference
- Local transport

UI Components Used

- st.sidebar
 - st.text_input
 - st.selectbox
 - st.multiselect
 - st.date_input
 - st.download_button
-

Prompt Engineering Strategy

The system uses structured prompt engineering for itinerary generation.

Prompt Sections

- User details
- Travel preferences
- Interests
- Budget constraints
- Special requests

Output Instructions

The AI is instructed to generate:

1. Flights
 2. Hotels
 3. Weather forecast
 4. Tourist attractions
 5. Day-wise itinerary
 6. Food suggestions
 7. Budget estimation
 8. Travel tips
-

Error Handling and Optimization

Several optimization techniques are implemented.

Error Handling

- Missing input validation
- Query parsing validation
- API retry handling
- Tool execution safeguards

Performance Optimizations

- Flash model usage
 - Reduced retries
 - JSON dataset storage
 - Modular architecture
-

Flight Search Tool

Purpose:

- Search flights between cities
- Recommend cheapest flights

Example:

```
flight_tool = Tool(  
    name="Flight Search Tool",  
    func=search_flights,  
    description="Search flights"  
)
```

Hotel Recommendation Tool

Purpose:

- Suggest hotels based on budget and ratings

Places Discovery Tool

Purpose:

- Recommend tourist attractions

Weather Lookup Tool

Purpose:

- Provide weather forecasts

Budget Estimation Tool

Purpose:

- Estimate complete travel expenses
-

User Interface

The project uses Streamlit to build an interactive web application.

Features of the UI:

- User-friendly interface
- Real-time AI responses
- Travel query input box
- Interactive recommendations
- Dynamic outputs

Example command:

```
streamlit run app.py
```

Sample Workflow

User Input

"Find cheapest flights from Delhi to Goa and recommend hotels under ₹3000."

AI Processing

1. AI identifies flight requirement.
2. AI selects Flight Search Tool.
3. AI identifies hotel requirement.
4. AI selects Hotel Recommendation Tool.
5. AI generates final response.

Output

- Cheapest flights
- Hotel recommendations
- Estimated budget

- Suggested attractions
-

Advantages of the System

- Saves time for users
 - Personalized recommendations
 - Intelligent decision-making
 - Automated travel planning
 - Natural language communication
 - Easy to use
 - Scalable architecture
-

Challenges Faced

During project development, several challenges were encountered:

1. API Model Compatibility

Issue:

- Gemini model compatibility errors occurred.

Solution:

- Updated model version to gemini-1.5-flash.

2. LangChain Deprecation Warnings

Issue:

- `initialize_agent()` deprecation warning.

Solution:

- Planned migration to newer agent constructors.

3. Tool Input Parsing Errors

Issue:

- Multi-parameter tool functions caused errors.

Solution:

- Modified tools to handle string and dictionary inputs.

4. API Quota Limits

Issue:

- Gemini API quota exhaustion.

Solution:

- Reduced retries and optimized requests.
-

Future Enhancements

The project can be further improved by adding:

- Real-time flight APIs
 - Live hotel booking systems
 - Google Maps integration
 - Voice-based interaction
 - Multi-language support
 - Personalized recommendation engine
 - Memory-enabled AI assistant
 - Payment gateway integration
 - Multi-agent architecture
 - RAG (Retrieval-Augmented Generation)
-

Applications

This system can be used in:

- Tourism industry
 - Travel agencies
 - Hotel booking systems
 - AI virtual assistants
 - Smart tourism platforms
 - Personalized travel recommendation systems
-

Conclusion

The "Agentic AI-Based Travel Planning Assistant Using LangChain" successfully demonstrates the implementation of an intelligent AI-powered travel planning system using modern Generative AI technologies.

The project integrates LangChain, Gemini API, Streamlit, and custom tools to create an agentic workflow capable of reasoning, tool selection, and dynamic response generation.

The assistant improves user experience by automating travel planning tasks and providing personalized recommendations through natural language interaction.

This project highlights the growing potential of AI agents in solving real-world problems and represents a strong foundation for future AI-powered tourism applications.

References

1. LangChain Documentation
 2. Google Gemini API Documentation
 3. Streamlit Documentation
 4. Python Official Documentation
 5. Generative AI Research Papers
 6. AI Agent Architecture Resources
-

Keywords

Agentic AI, LangChain, Gemini API, Travel Assistant, Generative AI, AI Agents, Streamlit, Tool Calling, Large Language Models, Intelligent Travel Planning